

Database Technologies - Self-Learning

MySQL Lab Exercises

1. Explain EXISTS vs IN performance comparison with an example.

```
kd1_99523_sandhya1> SELECT *
-> FROM emp
-> WHERE deptno IN (10, 20, 30);
```

empno	ename	job	mgr	hire	sal	comm	deptno
7369	SMITH	CLERK	7902	1980-12-17	800.00	NULL	20
7499	ALLEN	SALESMAN	7698	1981-02-20	1600.00	300.00	30
7521	WARD	SALESMAN	7698	1981-02-22	1250.00	500.00	30
7566	JONES	MANAGER	7839	1981-04-02	2975.00	NULL	20
7654	MARTIN	SALESMAN	7698	1981-09-28	1250.00	1400.00	30
7698	BLAKE	MANAGER	7839	1981-05-01	2850.00	NULL	30
7782	CLARK	MANAGER	7839	1981-06-09	2450.00	NULL	10
7788	SCOTT	ANALYST	7566	1982-12-09	3000.00	NULL	20
7839	KING	PRESIDENT	NULL	1981-11-17	4000.00	NULL	10
7844	TURNER	SALESMAN	7698	1981-09-08	1500.00	0.00	30
7876	ADAMS	CLERK	7788	1983-01-12	1100.00	NULL	20
7900	JAMES	CLERK	7698	1981-12-03	950.00	NULL	30
7902	FORD	ANALYST	7566	1981-12-03	3000.00	NULL	20
7934	MILLER	CLERK	7782	1982-01-23	1300.00	NULL	10
1000	NILESH	TRAINER	NULL	2004-01-01	2850.00	NULL	20

15 rows in set (0.01 sec)

```
kd1_99523_sandhya1> SELECT *
-> FROM emp e
-> WHERE EXISTS (
->   SELECT 1
->   FROM dept d
->   WHERE d.deptno = e.deptno
-> );
```

empno	ename	job	mgr	hire	sal	comm	deptno
7369	SMITH	CLERK	7902	1980-12-17	800.00	NULL	20
7499	ALLEN	SALESMAN	7698	1981-02-20	1600.00	300.00	30
7521	WARD	SALESMAN	7698	1981-02-22	1250.00	500.00	30
7566	JONES	MANAGER	7839	1981-04-02	2975.00	NULL	20
7654	MARTIN	SALESMAN	7698	1981-09-28	1250.00	1400.00	30
7698	BLAKE	MANAGER	7839	1981-05-01	2850.00	NULL	30
7782	CLARK	MANAGER	7839	1981-06-09	2450.00	NULL	10
7788	SCOTT	ANALYST	7566	1982-12-09	3000.00	NULL	20
7839	KING	PRESIDENT	NULL	1981-11-17	4000.00	NULL	10
7844	TURNER	SALESMAN	7698	1981-09-08	1500.00	0.00	30
7876	ADAMS	CLERK	7788	1983-01-12	1100.00	NULL	20
7900	JAMES	CLERK	7698	1981-12-03	950.00	NULL	30
7902	FORD	ANALYST	7566	1981-12-03	3000.00	NULL	20
7934	MILLER	CLERK	7782	1982-01-23	1300.00	NULL	10
1000	NILESH	TRAINER	NULL	2004-01-01	2850.00	NULL	20

15 rows in set (0.04 sec)

2. Implement a trigger to update the product price in cart, when price is updated in products table.

```
kd1_99523_sandhya1> INSERT INTO users (username, email)
-> VALUES
-> ('sandhya', 'sandhya@gmail.com'),
-> ('rahul', 'rahul@gmail.com');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

kd1_99523_sandhya1> INSERT INTO products (name, price, stock)
-> VALUES
-> ('Laptop', 50000, 10),
-> ('Mouse', 500, 20);
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

kd1_99523_sandhya1> SELECT * FROM products;
+----+-----+-----+-----+
| id | name  | price | stock |
+----+-----+-----+-----+
| 1  | Laptop | 50000.00 | 10 |
| 2  | Mouse | 500.00 | 20 |
+----+-----+-----+-----+
2 rows in set (0.00 sec)

kd1_99523_sandhya1> INSERT INTO cart (user_id, product_id, price, quantity)
-> VALUES
-> (1, 1, 50000, 1),
-> (2, 2, 500, 2);
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

kd1_99523_sandhya1> UPDATE products
-> SET price = 55000
-> WHERE id = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

```
kd1_99523_sandhya1> INSERT INTO users (username, email)
-> VALUES
-> ('sandhya', 'sandhya@gmail.com'),
-> ('rahul', 'rahul@gmail.com');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

kd1_99523_sandhya1> INSERT INTO products (name, price, stock)
-> VALUES
-> ('Laptop', 50000, 10),
-> ('Mouse', 500, 20);
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

kd1_99523_sandhya1> SELECT * FROM products;
+----+-----+-----+-----+
| id | name  | price | stock |
+----+-----+-----+-----+
| 1  | Laptop | 50000.00 | 10 |
| 2  | Mouse | 500.00 | 20 |
+----+-----+-----+-----+
2 rows in set (0.00 sec)

kd1_99523_sandhya1> INSERT INTO cart (user_id, product_id, price, quantity)
-> VALUES
-> (1, 1, 50000, 1),
-> (2, 2, 500, 2);
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

kd1_99523_sandhya1> UPDATE products
-> SET price = 55000
-> WHERE id = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

```

kd1_99523_sandhya1> DELIMITER $$
kd1_99523_sandhya1>
kd1_99523_sandhya1> CREATE TRIGGER trg_product_price_update
-> AFTER UPDATE ON products
-> FOR EACH ROW
-> BEGIN
->     UPDATE cart
->     SET price = NEW.price
->     WHERE product_id = NEW.id;
-> END$$
Query OK, 0 rows affected (0.01 sec)

kd1_99523_sandhya1>
kd1_99523_sandhya1> DELIMITER ;

```

```

kd1_99523_sandhya1> UPDATE dept
-> SET deptno = 11
-> WHERE deptno = 10;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

```

```

kd1_99523_sandhya1> SELECT * FROM emp;
+-----+-----+-----+-----+-----+-----+-----+-----+
| empno | ename  | job      | mgr  | hire      | sal      | comm      | deptno |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 7369 | SMITH  | CLERK    | 7902 | 1980-12-17 | 800.00   | NULL      | 20     |
| 7499 | ALLEN  | SALESMAN | 7698 | 1981-02-20 | 1600.00  | 300.00    | 30     |
| 7521 | WARD   | SALESMAN | 7698 | 1981-02-22 | 1250.00  | 500.00    | 30     |
| 7566 | JONES  | MANAGER  | 7839 | 1981-04-02 | 2975.00  | NULL      | 20     |
| 7654 | MARTIN | SALESMAN | 7698 | 1981-09-28 | 1250.00  | 1400.00   | 30     |
| 7698 | BLAKE  | MANAGER  | 7839 | 1981-05-01 | 2850.00  | NULL      | 30     |
| 7782 | CLARK  | MANAGER  | 7839 | 1981-06-09 | 2450.00  | NULL      | 10     |
| 7788 | SCOTT  | ANALYST  | 7566 | 1982-12-09 | 3000.00  | NULL      | 20     |
| 7839 | KING   | PRESIDENT | NULL | 1981-11-17 | 4000.00  | NULL      | 10     |
| 7844 | TURNER | SALESMAN | 7698 | 1981-09-08 | 1500.00  | 0.00      | 30     |
| 7876 | ADAMS  | CLERK    | 7788 | 1983-01-12 | 1100.00  | NULL      | 20     |
| 7900 | JAMES  | CLERK    | 7698 | 1981-12-03 | 950.00   | NULL      | 30     |
| 7902 | FORD   | ANALYST  | 7566 | 1981-12-03 | 3000.00  | NULL      | 20     |
| 7934 | MILLER | CLERK    | 7782 | 1982-01-23 | 1300.00  | NULL      | 10     |
| 1000 | NILESH | TRAINER  | NULL | 2004-01-01 | 2850.00  | NULL      | 20     |
+-----+-----+-----+-----+-----+-----+-----+-----+
15 rows in set (0.00 sec)

```

- Understand ON DELETE CASCADE, ON UPDATE CASCADE. Write emp, dept CREATE TABLE statements and apply CASCADE for all foreign keys

```

CREATE TABLE emp (
    empno INT PRIMARY KEY,
    ename VARCHAR(50) NOT NULL,
    job VARCHAR(50),
    sal DECIMAL(10,2),
    deptno INT,

    CONSTRAINT fk_emp_dept
    FOREIGN KEY (deptno)
    REFERENCES dept(deptno)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);

```

```
INSERT INTO dept VALUES  
(10, 'ACCOUNTING', 'PUNE'),  
(20, 'RESEARCH', 'MUMBAI'),  
(30, 'SALES', 'NASHIK');
```

```
INSERT INTO emp VALUES  
(101, 'Amit', 'CLERK', 1200, 10),  
(102, 'Neha', 'MANAGER', 3500, 10),  
(103, 'Ravi', 'ANALYST', 2800, 20),  
(104, 'Pooja', 'CLERK', 1500, 30);
```

ON UPDATE CASCADE

```
UPDATE dept  
SET deptno = 11  
WHERE deptno = 10;
```

ON DELETE CASCADE

```
DELETE FROM dept  
WHERE deptno = 20
```

MongoDb Lab Exercises

1. Practice CRUD operations on Mongo.
 1. Create two collections -- emp and dept.
 2. Insert the emp and dept records.
 3. Increase salary of dept 10 emps by 100.
 4. Display emp name, job, deptno, sal in sorted order of deptno, job, sal (desc) for all emps whose sal more than 1000.
 5. Delete the dept with id 40.
-
2. Learn grouping operations using Mongo aggregation pipeline
 1. Display job wise total sal, max sal, min sal, and avg sal for all jobs other than PRESIDENT.
 2. Display emp name, dept name, income (sal + comm) in sorted order of their income.

```

mongosh> use cdacdb
switched to db cdacdb
cdacdb> db.dept.insertMany([
...   {
...     deptno: 10,
...     dname: "ACCOUNTING",
...     loc: "PUNE"
...   },
...   {
...     deptno: 20,
...     dname: "RESEARCH",
...     loc: "MUMBAI"
...   },
...   {
...     deptno: 30,
...     dname: "SALES",
...     loc: "NASHIK"
...   },
...   {
...     deptno: 40,
...     dname: "OPERATIONS",
...     loc: "DELHI"
...   }
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6aa7ee9dd14013632d6c4bd0'),
    '1': ObjectId('6aa7ee9dd14013632d6c4bd1'),
    '2': ObjectId('6aa7ee9dd14013632d6c4bd2'),
    '3': ObjectId('6aa7ee9dd14013632d6c4bd3')
  }
}

```

```

cdacdb> db.emp.insertMany([
...   {
...     empno: 7369,
...     ename: "SMITH",
...     job: "CLERK",
...     sal: 800,
...     deptno: 20
...   },
...   {
...     empno: 7499,
...     ename: "ALLEN",
...     job: "SALESMAN",
...     sal: 1600,
...     deptno: 30
...   },
...   {
...     empno: 7521,
...     ename: "WARD",
...     job: "SALESMAN",
...     sal: 1250,
...     deptno: 30
...   },
...   {
...     empno: 7566,
...     ename: "JONES",
...     job: "MANAGER",
...     sal: 2975,
...     deptno: 20
...   },
...   {
...     empno: 7839,
...     ename: "KING",
...     job: "PRESIDENT",
...     sal: 5000,
...     deptno: 10
...   },
...   {
...     empno: 7900,
...     ename: "JAMES",
...     job: "CLERK",
...     sal: 950,
...     deptno: 30
...   }
... ])

```

```
cdacdb> db.emp.find()
[
  {
    _id: ObjectId('6aa7eea9d14013632d6c4bd4'),
    empno: 7369,
    ename: 'SMITH',
    job: 'CLERK',
    sal: 800,
    deptno: 20
  },
  {
    _id: ObjectId('6aa7eea9d14013632d6c4bd5'),
    empno: 7499,
    ename: 'ALLEN',
    job: 'SALESMAN',
    sal: 1600,
    deptno: 30
  },
  {
    _id: ObjectId('6aa7eea9d14013632d6c4bd6'),
    empno: 7521,
    ename: 'WARD',
    job: 'SALESMAN',
    sal: 1250,
    deptno: 30
  },
  {
    _id: ObjectId('6aa7eea9d14013632d6c4bd7'),
    empno: 7566,
    ename: 'JONES',
    job: 'MANAGER',
    sal: 2975,
    deptno: 20
  },
  {
    _id: ObjectId('6aa7eea9d14013632d6c4bd8'),
    empno: 7839,
    ename: 'KING',
    job: 'PRESIDENT',
    sal: 5000,
  }
]
```

```
cdacdb> db.emp.updateMany(
...   { deptno: 10 },
...   { $inc: { sal: 100 } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
```

```
cdacdb> db.emp.find(
...   { sal: { $gt: 1000 } },
...   {
...     _id: 0,
...     ename: 1,
...     job: 1,
...     deptno: 1,
...     sal: 1
...   }
... ).sort({
...   deptno: 1,
...   job: 1,
...   sal: -1
... })
[
  { ename: 'MILLER', job: 'CLERK', sal: 1400, deptno: 10 },
  { ename: 'CLARK', job: 'MANAGER', sal: 2550, deptno: 10 },
  { ename: 'KING', job: 'PRESIDENT', sal: 5100, deptno: 10 },
  { ename: 'JONES', job: 'MANAGER', sal: 2975, deptno: 20 },
  { ename: 'ALLEN', job: 'SALESMAN', sal: 1600, deptno: 30 },
  { ename: 'WARD', job: 'SALESMAN', sal: 1250, deptno: 30 }
]
```

```

cdacdb> db.dept.deleteOne({
...   deptno: 40
... })
{ acknowledged: true, deletedCount: 1 }
cdacdb> db.dept.find()
[
  {
    _id: ObjectId('6aa7ee9dd14013632d6c4bd0'),
    deptno: 10,
    dname: 'ACCOUNTING',
    loc: 'PUNE'
  },
  {
    _id: ObjectId('6aa7ee9dd14013632d6c4bd1'),
    deptno: 20,
    dname: 'RESEARCH',
    loc: 'MUMBAI'
  },
  {
    _id: ObjectId('6aa7ee9dd14013632d6c4bd2'),
    deptno: 30,
    dname: 'SALES',
    loc: 'NASHIK'
  }
]

```

```

cdacdb> db.emp.aggregate([
...   {
...     $match: {
...       job: { $ne: "PRESIDENT" }
...     },
...   },
...   {
...     $group: {
...       _id: "$job",
...       total_sal: {
...         $sum: "$sal"
...       },
...       max_sal: {
...         $max: "$sal"
...       },
...       min_sal: {
...         $min: "$sal"
...       },
...       avg_sal: {
...         $avg: "$sal"
...       }
...     }
...   },
...   {
...     $project: {
...       _id: 0,
...       job: "$_id",
...       total_sal: 1,
...       max_sal: 1,
...       min_sal: 1,
...       avg_sal: 1
...     }
...   }
... ])

```

```
cdacdb> db.emp.aggregate([
...   {
...     $lookup: {
...       from: "dept",
...       localField: "deptno",
...       foreignField: "deptno",
...       as: "department"
...     }
...   },
...   {
...     $unwind: "$department"
...   },
...   {
...     $project: {
...       _id: 0,
...       ename: 1,
...       dname: "$department.dname",
...
...       income: {
...         $add: [
...           "$sal",
...           { $ifNull: ["$comm", 0] }
...         ]
...       }
...     }
...   },
...   {
...     $sort: {
...       income: 1
...     }
...   }
... ])
[
  { ename: 'SMITH', dname: 'RESEARCH', income: 800 },
  { ename: 'JAMES', dname: 'SALES', income: 950 },
  { ename: 'WARD', dname: 'SALES', income: 1250 },
  { ename: 'MILLER', dname: 'ACCOUNTING', income: 1400 },
```

Leetcode Exercises

Problem List

<

>

MySQL

Submit

Premium

DescriptionEditorialSolutionsSubmissions

586. Customer Placing the Largest Number of Orders

EasyTopicsCompaniesHint

SQL SchemaPandas Schema

Table: Orders

Column Name	Type
order_number	int
customer_number	int

order_number is the primary key (column with unique values) for this table.
This table contains information about the order ID and the customer ID.

Write a solution to find the customer_number for the customer who has placed the largest number of orders.

The test cases are generated so that exactly one customer will have placed more orders than any other customer.

1.3K11726 Online

Code

MySQLAuto

```
1 # Write your MySQL query statement below
2 SELECT customer_number
3 FROM Orders
4 GROUP BY customer_number
5 ORDER BY COUNT(*) DESC
6 LIMIT 1;
7
```

SavedLn 7, Col 1

TestcaseTest Result

AcceptedRuntime: 88 ms

Case 1

Input

orders =

order_number	customer_number
1	1
2	2
3	3

[illegible]

Problem List < > ✕

Accepted × | Editorial | Solutions | Submissions

All Submissions

Accepted 11 / 11 testcases passed

Sandhya Mandlik submitted at Sep 09, 2026 22:46

Unlock the Full LeetCode Experience

₹583.25/mo Save over 98%

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

401 ms Beats 24.23%

Testcase | Test Result

Accepted Runtime: 69 ms

Case 1

Input

Courses =

student	class
A	Math
B	English
C	Math

Code

```
MySQL
1 # Write your MySQL query statement below
2 SELECT class
3 FROM Courses
4 GROUP BY class
5 HAVING COUNT(student) >= 5;
```

Saved Ln 5, Col 28

JOINS Problems

Problem List < > ✕

Accepted × | Editorial | Solutions | Submissions

All Submissions

Accepted 25 / 25 testcases passed

Sandhya Mandlik submitted at Sep 09, 2026 22:53

Unlock the Full LeetCode Experience

₹583.25/mo Save over 98%

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

1257 ms Beats 70.48%

Testcase | Test Result

Expected

unique_id	name
null	Alice
null	Bob
2	Meir
3	Winston
1	Jonathan

Code

```
MySQL
1 # Write your MySQL query statement below
2 SELECT
3   eu.unique_id,
4   e.name
5 FROM Employees e
6 LEFT JOIN EmployeeUNI eu
7   ON e.id = eu.id;
```

Saved Ln 7, Col 21

Problem List < > ✕

Accepted × | Editorial | Solutions | Submissions

All Submissions

Accepted 12 / 12 testcases passed

Sandhya Mandlik submitted at Sep 09, 2026 22:55

Unlock the Full LeetCode Experience

₹583.25/mo Save over 98%

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

779 ms Beats 15.45%

Testcase | Test Result

Accepted Runtime: 119 ms

Case 1

Input

Customers =

id	name
1	Joe
2	Henry
3	Sam

Code

```
MySQL
1 # Write your MySQL query statement below
2 SELECT c.name AS Customers
3 FROM Customers c
4 LEFT JOIN Orders o
5   ON c.id = o.customerId
6 WHERE o.customerId IS NULL;
```

Saved Ln 6, Col 28

Problem List

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 13 / 13 testcases passed

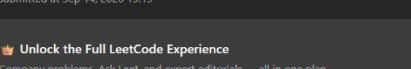
Sandhya Mandlik submitted at Sep 14, 2026 19:19

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

1343 ms | Beats 58.79%



Code

MySQL | Auto

```
1 # Write your MySQL query statement below
2 SELECT p.product_name,
3        s.year,
4        s.price
5 FROM Sales s
6 JOIN Product p
7 ON s.product_id = p.product_id;
```

Saved

Ln 7, Col 32

Testcase | Test Result

Accepted Runtime: 147 ms

Case 1

Input

Sales =

sale_id	product_id	year	quantity	price
1	100	2008	10	5000
2	100	2009	12	5000
7	200	2011	15	9000

Problem List

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted 11 / 11 testcases passed

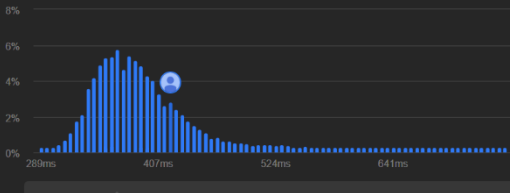
Sandhya Mandlik submitted at Sep 14, 2026 19:21

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

420 ms Beats 28.52%



Code

MySQL Auto

```
1 # Write your MySQL query statement below
2 SELECT e.name
3 FROM Employee e
4 JOIN Employee r
5 ON e.id = r.managerId
6 GROUP BY e.id, e.name
7 HAVING COUNT(r.id) >= 5;
```

Testcase Test Result

Accepted Runtime: 89 ms

Case 1

Input

id	name	department	managerId
101	John	A	null
102	Dan	A	101
103	Jane	A	101

HackerRank

Prepare

Certify

Compete

Search

Rank: 3059997 | Points: 10/80

Prepare > SQL > Basic Join > Population Census > Submissions

Population Census

70 more points to get your first star!

Problem

Submissions

Leaderboard

Discussions

RESULT	SCORE	LANGUAGE	TIME
Accepted	10.0	MySQL	a minute ago

NEED HELP?

View discussions

View top submissions

1

Problem List < > Submit

Description | Accepted x | Editorial | Solutions | Submissions

All Submissions

Accepted 29 / 29 testcases passed

Sandhya Mandlik submitted at Sep 14, 2026 19:26

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

1816 ms | Beats 15.15%

Testcase | Test Result

Accepted Runtime: 69 ms

Case 1

Input

Queue =

person_id	person_name	weight	turn
5	Alice	250	1
4	Bob	175	5
3	Alex	350	2

Problem List < > Submit

Description | Accepted x | Editorial | Solutions | Submissions

All Submissions

Accepted 16 / 16 testcases passed

Sandhya Mandlik submitted at Sep 14, 2026 19:26

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

790 ms | Beats 19.30%

Testcase | Test Result

Accepted Runtime: 83 ms

Case 1

Input

Products =

product_id	new_price	change_date
1	20	2019-08-14
2	50	2019-08-14
1	30	2019-08-15

Subquery Problems

Problem List < > Submit

Description | Accepted x | Editorial | Solutions | Submissions

All Submissions

Accepted 14 / 14 testcases passed

Sandhya Mandlik submitted at Sep 14, 2026 20:11

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

447 ms | Beats 38.93%

Testcase | Test Result

Accepted Runtime: 82 ms

Case 1

Input

Employee =

id	name	salary	managerId
1	Joc	70000	3
2	Henry	80000	4
3	Sam	60000	null

Accepted 15 / 15 testcases passed
Sandhya Mandlik submitted at Sep 14, 2026 20:57

Unlock the Full LeetCode Experience
 Company problems, Ask Leet, and expert editors — all in one plan.

Runtime
 495 ms | Beats 70.15%

Code

```
MySQL ▾ Auto
1 # Write your MySQL query statement below
2 SELECT w1.id
3 FROM Weather w1
4 JOIN Weather w2
5 ON DATEDIFF(w1.recordDate, w2.recordDate) = 1
6 WHERE w1.temperature > w2.temperature;
```

Saved Ln 6, Col 39

Testcase | Test Result

Accepted Runtime: 63 ms

Case 1

Input

id	recordDate	temperature
1	2015-01-01	10
2	2015-01-02	25
3	2015-01-03	20

Problem List < > Submit

Description Accepted x Editorial Solutions Submissions

All Submissions

Accepted 21 / 21 testcases passed
Sandhya Mandlik submitted at Sep 14, 2026 20:59

Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime
1638 ms | Beats 47.68%

Testcase | Test Result
Accepted Runtime: 145 ms

Case 1

Input

SalesPerson =

sales_id	name	salary	commission_rate	hire_date
1	John	100000	6	4/1/2006
2	Amy	12000	5	5/1/2010
3	Mark	65000	12	12/25/2008

Code

```
MySQL
4 WHERE s.sales_id NOT IN (
5   SELECT o.sales_id
6   FROM Orders o
7   JOIN Company c
8   ON o.com_id = c.com_id
9   WHERE c.name = 'RED'
10 );
```

Saved Ln 10, Col 3

Problem List < > Submit

Description Accepted x Editorial Solutions Submissions

All Submissions

Accepted 23 / 23 testcases passed
Sandhya Mandlik submitted at Sep 14, 2026 21:00

Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime
537 ms | Beats 93.05%

Testcase | Test Result
Accepted Runtime: 139 ms

Case 1

Input

Logs =

id	num
1	1
2	1
3	1

Code

```
MySQL
3 FROM Logs l1
4 JOIN Logs l2
5 ON l2.id = l1.id + 1
6 JOIN Logs l3
7 ON l3.id = l1.id + 2
8 WHERE l1.num = l2.num
9 AND l2.num = l3.num;
```

Saved Ln 9, Col 21

Problem List < > Submit

Description Accepted x Editorial Solutions Submissions

All Submissions

Accepted 8 / 8 testcases passed
Sandhya Mandlik submitted at Sep 14, 2026 21:01

Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime
450 ms | Beats 58.43%

Testcase | Test Result
Accepted Runtime: 110 ms

Case 1

Input

Person =

personId	lastName	firstName
1	Wang	Allen
2	Alice	Bob

Code

```
MySQL
2 SELECT p.firstName,
3       p.lastName,
4       a.city,
5       a.state
6 FROM Person p
7 LEFT JOIN Address a
8 ON p.personId = a.personId;
```

Saved Ln 8, Col 28

Problem List

Description

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted 22 / 22 testcases passed

Sandhya Mandlik submitted at Sep 14, 2026 21:03

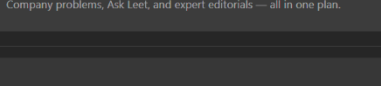
583.25ms

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

892 ms | Beat: 63.54%



Code

MySQL

Auto

```
1 # Write your MySQL query statement below
2 DELETE p1
3 FROM Person p1
4 JOIN Person p2
5 ON p1.email = p2.email
6 AND p1.id > p2.id;
```

Saved

In 6, Col 1

Testcase

Test Result

Accepted Runtime: 66 ms

Case 1

Input

Person

	id	email
1	john@example.com	
2	bob@example.com	
3	johnd@example.com	

Problem List

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted

29 / 29 testcases passed

Sandhya Mandlik submitted at Sep 14, 2026 21:04

Solution

₹583.25/mo

Save over 98%

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

2046 ms | Beats: 7.54%

Code

MySQL

Auto

```
1 # Write your MySQL query statement below
2 SELECT person_name
3 FROM Queue q
4 WHERE (
5     SELECT SUM(weight)
6     FROM Queue q2
7     WHERE q2.turn <= q.turn
8 ) <= 1000
```

Saved

Ln 1, Col 1

Testcase

Test Result

Accepted

Runtime: 70 ms

Case 1

Input

Queue =

person_id	person_name	weight	turn
5	Alice	250	1
4	Bob	175	5
3	Alex	350	2

Problem List

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted

10 / 10 testcases passed

Sandhya Mandlik submitted at Sep 14, 2026 21:04

Solution

₹583.25/mo

Save over 98%

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

263 ms | Beats: 94.30%

Code

MySQL

Auto

```
1 # Write your MySQL query statement below
2 SELECT (
3     SELECT DISTINCT salary
4     FROM Employee
5     ORDER BY salary DESC
6     LIMIT 1 OFFSET 1
7 ) AS SecondHighestSalary;
```

Saved

Ln 7, Col 25

Testcase

Test Result

Accepted

Runtime: 115 ms

Case 1

Case 2

Input

Employee =

id	salary
1	100
2	200
3	300

CDAC Admission Process Case Study

- Design database for CDAC Admission System.
- Create appropriate tables and insert sample records. Write a Stored Procedure for Round1 allocation.
- Write a Stored Procedure for Round2 allocation.

```
kdl_99523_sandhya1> CREATE DATABASE cdac_admission;
Query OK, 1 row affected (0.02 sec)

kdl_99523_sandhya1>
kdl_99523_sandhya1> USE cdac_admission;
Database changed
kdl_99523_sandhya1> CREATE TABLE candidate (
->   candidate_id INT PRIMARY KEY AUTO_INCREMENT,
->   name VARCHAR(100) NOT NULL,
->   email VARCHAR(120) NOT NULL UNIQUE,
->   phone VARCHAR(15) UNIQUE,
->   cc_score INT NOT NULL CHECK (cc_score >= 0),
->   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
-> );
Query OK, 0 rows affected (0.07 sec)

kdl_99523_sandhya1> CREATE TABLE course (
->   course_id INT PRIMARY KEY AUTO_INCREMENT,
->   course_name VARCHAR(100) NOT NULL UNIQUE
-> );
Query OK, 0 rows affected (0.04 sec)

kdl_99523_sandhya1> ^C
kdl_99523_sandhya1> CREATE TABLE center (
->   center_id INT PRIMARY KEY AUTO_INCREMENT,
->   center_name VARCHAR(100) NOT NULL,
->   city VARCHAR(50) NOT NULL
-> );
Query OK, 0 rows affected (0.03 sec)

kdl_99523_sandhya1> CREATE TABLE center_course (
->   center_id INT NOT NULL,
->   course_id INT NOT NULL,
```

```
->   capacity INT NOT NULL CHECK (capacity >= 0),
->   PRIMARY KEY (center_id, course_id),
->   FOREIGN KEY (center_id)
->     REFERENCES center(center_id)
->     ON DELETE CASCADE
->     ON UPDATE CASCADE,
->   FOREIGN KEY (course_id)
->     REFERENCES course(course_id)
->     ON DELETE CASCADE
->     ON UPDATE CASCADE
-> );
Query OK, 0 rows affected (0.07 sec)

kdl_99523_sandhya1> CREATE TABLE candidate_preference (
->   candidate_id INT NOT NULL,
->   preference_no INT NOT NULL
->     CHECK (preference_no > 0),
->   center_id INT NOT NULL,
->   course_id INT NOT NULL,
->   PRIMARY KEY (candidate_id, preference_no),
->   FOREIGN KEY (candidate_id)
->     REFERENCES candidate(candidate_id)
->     ON DELETE CASCADE
->     ON UPDATE CASCADE,
->   FOREIGN KEY (center_id, course_id)
->     REFERENCES center_course(center_id, course_id)
->     ON DELETE RESTRICT
->     ON UPDATE CASCADE
-> );
Query OK, 0 rows affected (0.06 sec)
```

```

kdl_99523_sandhyal> CREATE TABLE allocation (
->     allocation_id INT PRIMARY KEY AUTO_INCREMENT,
->
->     candidate_id INT NOT NULL UNIQUE,
->
->     center_id INT NOT NULL,
->
->     course_id INT NOT NULL,
->
->     round_no INT NOT NULL
->         CHECK (round_no IN (1,2)),
->
->     allocated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
->
->     FOREIGN KEY (candidate_id)
->         REFERENCES candidate(candidate_id)
->         ON DELETE CASCADE
->         ON UPDATE CASCADE,
->
->     FOREIGN KEY (center_id, course_id)
->         REFERENCES center_course(center_id, course_id)
->         ON DELETE RESTRICT
->         ON UPDATE CASCADE
-> );
Query OK, 0 rows affected (0.07 sec)

kdl_99523_sandhyal> INSERT INTO course(course_name)
-> VALUES
-> ('DBDA'),
-> ('DAC'),
-> ('DITIIS');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

kdl_99523_sandhyal> INSERT INTO center(center_name, city)
-> VALUES
-> ('Pune Center', 'Pune'),
-> ('Mumbai Center', 'Mumbai'),
-> ('Nashik Center', 'Nashik');
Query OK, 3 rows affected (0.01 sec)

```

```

kdl_99523_sandhyal> INSERT INTO center_course
-> (center_id, course_id, capacity)
-> VALUES
-> (1,1,2),
-> (1,2,2),
-> (2,1,2),
-> (2,2,2),
-> (3,1,2);
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0

kdl_99523_sandhyal> INSERT INTO candidate
-> (name, email, phone, cc_score)
-> VALUES
-> ('Asha', 'asha@example.com', '9000000001', 850),
-> ('Riya', 'riya@example.com', '9000000002', 820),
-> ('Neha', 'neha@example.com', '9000000003', 790),
-> ('Kiran', 'kiran@example.com', '9000000004', 760);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

kdl_99523_sandhyal> INSERT INTO candidate_preference VALUES
-> (1,1,1,1),
-> (1,2,2,1);
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

kdl_99523_sandhyal> INSERT INTO candidate_preference VALUES
-> (2,1,1,2),
-> (2,2,2,1);
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

kdl_99523_sandhyal> INSERT INTO candidate_preference VALUES
-> (4,1,2,1),
-> (4,2,3,1);
Query OK, 2 rows affected (0.00 sec)
Records: 2 Duplicates: 0 Warnings: 0

```

```

kdl_99523_sandhyal> DELIMITER $$
kdl_99523_sandhyal>
kdl_99523_sandhyal> CREATE PROCEDURE round1_allocation(
->     OUT p_message VARCHAR(255)
-> )
-> BEGIN
->
->     DECLARE done INT DEFAULT 0;
->
->     DECLARE v_candidate_id INT;
->
->     DECLARE v_center_id INT;
->
->     DECLARE v_course_id INT;
->
->     DECLARE v_allocated INT DEFAULT 0;
->
->
->     DECLARE cur CURSOR FOR
->
->         SELECT candidate_id
->         FROM candidate
->         ORDER BY cc_score DESC, candidate_id;
->
->
->     DECLARE CONTINUE HANDLER
->     FOR NOT FOUND
->     SET done = 1;
->
->
->     DECLARE EXIT HANDLER
->     FOR SQLEXCEPTION
->     BEGIN
->
->         ROLLBACK;
->
->         SET p_message =
->         'Round1 failed - transaction rolled back';
->
->     END;

```

```

-> START TRANSACTION;
->
->
-> OPEN cur;
->
->
-> read_loop: LOOP
->
->     FETCH cur
->     INTO v_candidate_id;
->
->     IF done = 1 THEN
->         LEAVE read_loop;
->
->     END IF;
->
->     SELECT cp.center_id,
->            cp.course_id
->
->     INTO v_center_id,
->            v_course_id
->
->     FROM candidate_preference cp
->
->     WHERE cp.candidate_id = v_candidate_id
->
->     AND cp.preference_no = 1
->
->     AND (
->         SELECT COUNT(*)
->         FROM allocation a
->         WHERE a.center_id = cp.center_id
->         AND a.course_id = cp.course_id
->     )
->     <
->     (
->         SELECT capacity
->         FROM center_course cc

```

```

->
->     LIMIT 1;
->
->     IF v_center_id IS NOT NULL THEN
->
->         INSERT IGNORE INTO allocation
->         (
->             candidate_id,
->             center_id,
->             course_id,
->             round_no
->         )
->
->         VALUES
->         (
->             v_candidate_id,
->             v_center_id,
->             v_course_id,
->             1
->         );
->
->         SET v_allocated =
->             v_allocated + 1;
->
->     END IF;
->
->     SET v_center_id = NULL;
->
->     SET v_course_id = NULL;
->
->
-> END LOOP;
->
->
-> CLOSE cur;
->
-> COMMIT;

```

```

->
->     LIMIT 1;
->
->     IF v_center_id IS NOT NULL THEN
->
->         INSERT IGNORE INTO allocation
->         (
->             candidate_id,
->             center_id,
->             course_id,
->             round_no
->         )
->
->         VALUES
->         (
->             v_candidate_id,
->             v_center_id,
->             v_course_id,
->             1
->         );
->
->         SET v_allocated =
->             v_allocated + 1;
->
->     END IF;
->
->     SET v_center_id = NULL;
->
->     SET v_course_id = NULL;
->
->
-> END LOOP;
->
->
-> CLOSE cur;
->
-> COMMIT;

```

```

-> SET p_message =
-> CONCAT(
-> 'Round1 completed. Allocations attempted: ',
-> v_allocated
-> );
-> END$$

```

Query OK, 0 rows affected (0.01 sec)

```

kd1_99523_sandhya1>
kd1_99523_sandhya1> DELIMITER ;
kd1_99523_sandhya1> CALL round1_allocation(@msg);
Query OK, 0 rows affected (0.03 sec)

```

```

kd1_99523_sandhya1> SELECT @msg;

```

```

+-----+
| @msg                                     |
+-----+
| Round1 completed. Allocations attempted: 2 |
+-----+
1 row in set (0.00 sec)

```

```

kd1_99523_sandhya1> SELECT * FROM allocation;

```

```

+-----+-----+-----+-----+-----+-----+
| allocation_id | candidate_id | center_id | course_id | round_no | allocated_at |
+-----+-----+-----+-----+-----+-----+
| 1             | 1            | 1         | 1         | 1        | 2026-09-14 21:11:19 |
| 2             | 2            | 1         | 2         | 1        | 2026-09-14 21:11:19 |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

```

```

kd1_99523_sandhya1> DELIMITER $$

```

```

kd1_99523_sandhya1>
kd1_99523_sandhya1> CREATE PROCEDURE round2_allocation(
-> IN p_round INT,
-> INOUT p_allocated_count INT,
-> OUT p_message VARCHAR(255)
-> )
-> BEGIN

```

```

-> DECLARE done INT DEFAULT 0;
-> DECLARE v_candidate_id INT;
-> DECLARE v_center_id INT;
-> DECLARE v_course_id INT;
-> DECLARE v_pref_no INT DEFAULT 2;
-> DECLARE v_has_capacity INT DEFAULT 0;
->
-> DECLARE cur CURSOR FOR
-> SELECT c.candidate_id
-> FROM candidate c
-> LEFT JOIN allocation a
-> ON a.candidate_id = c.candidate_id
-> WHERE a.candidate_id IS NULL
-> ORDER BY c.cc_score DESC,
-> c.candidate_id;
->
-> DECLARE CONTINUE HANDLER
-> FOR NOT FOUND
-> SET done = 1;
->
-> DECLARE EXIT HANDLER
-> FOR SQLEXCEPTION
-> BEGIN
-> ROLLBACK;
-> SET p_message =

```

```

-> SET p_message =
-> 'Round2 failed - transaction rolled back';
-> END;
->
-> IF p_round <> 2 THEN
-> SET p_message =
-> 'Invalid round number';
-> ELSE
-> START TRANSACTION;
-> OPEN cur;
->
-> candidate_loop: LOOP
-> FETCH cur
-> INTO v_candidate_id;
->
-> IF done = 1 THEN
-> LEAVE candidate_loop;
-> END IF;
->
-> SET v_pref_no = 2;
->
-> WHILE v_pref_no <= 10 DO
->
-> SET v_center_id = NULL;
-> SET v_course_id = NULL;

```

```

->      INTO v_center_id,
->          v_course_id
->
->      FROM candidate_preference
->
->      WHERE candidate_id =
->            v_candidate_id
->
->      AND preference_no =
->            v_pref_no
->
->      LIMIT 1;
->
->      IF v_center_id IS NULL THEN
->
->          SET v_pref_no = 11;
->
->      ELSE
->
->          SELECT CASE
->
->              WHEN
->              (
->                  SELECT COUNT(*)
->                  FROM allocation a
->
->                  WHERE a.center_id =
->                        v_center_id
->
->                  AND a.course_id =
->                        v_course_id
->              )
->              <
->              (
->                  SELECT capacity
->                  FROM center_course cc
->
->                  WHERE cc.center_id =
->                        v_center_id

```

```

->          v_center_id
->
->          AND cc.course_id =
->                v_course_id
->          )
->
->      THEN 1
->
->      ELSE 0
->
->      END
->
->      INTO v_has_capacity;
->
->      IF v_has_capacity = 1 THEN
->
->          INSERT INTO allocation
->          (
->              candidate_id,
->              center_id,
->              course_id,
->              round_no
->          )
->
->          VALUES
->          (
->              v_candidate_id,
->              v_center_id,
->              v_course_id,
->              2
->          );
->
->
->          SET p_allocated_count =
->                p_allocated_count + 1;
->
->
->          SET v_pref_no = 11;

```

```

->
->
->         ELSE
->
->             SET v_pref_no =
->                 v_pref_no + 1;
->
->         END IF;
->
->     END IF;
->
->     END WHILE;
->
-> END LOOP;
->
-> CLOSE cur;
->
-> COMMIT;
->
->
->     SET p_message =
->         'Round2 completed successfully';
->
-> END IF;
->
-> END$$
Query OK, 0 rows affected (0.02 sec)

kd1_99523_sandhya1>
kd1_99523_sandhya1> DELIMITER ;
kd1_99523_sandhya1> SET @cnt = 0;
Query OK, 0 rows affected (0.01 sec)

kd1_99523_sandhya1>
kd1_99523_sandhya1> CALL round2_allocation(
->     2,
->     @cnt,
->     @msg
-> );
Query OK, 0 rows affected (0.00 sec)

```

```

kd1_99523_sandhya1>
kd1_99523_sandhya1> DELIMITER ;
kd1_99523_sandhya1> SET @cnt = 0;
Query OK, 0 rows affected (0.01 sec)

kd1_99523_sandhya1>
kd1_99523_sandhya1> CALL round2_allocation(
->     2,
->     @cnt,
->     @msg
-> );
Query OK, 0 rows affected (0.00 sec)

kd1_99523_sandhya1>
kd1_99523_sandhya1> SELECT @cnt;
+-----+
| @cnt |
+-----+
| 0 |
+-----+
1 row in set (0.00 sec)

kd1_99523_sandhya1>
kd1_99523_sandhya1> SELECT @msg;
+-----+
| @msg |
+-----+
| Round2 completed successfully |
+-----+
1 row in set (0.00 sec)

kd1_99523_sandhya1> |

```